

```

#include <stdio.h>
#include <stdlib.h>
struct node {
int data;
struct node *next;
};
struct node *head = NULL;
// Function to create a new node struct node* createNode(int data) {
struct node *newNode = (struct node*)malloc(sizeof(struct node)); newNode->data = data;
newNode->next = NULL; return newNode;
}
// Function to create a circular linked list void createList(int data) {
head = createNode(data);
head->next = head; // Make it circular
}
// Function to insert a node at the beginning void insertAtBeginning(int data) {
struct node *newNode = createNode(data);
if (head == NULL) {
head = newNode; head->next = head;
} else {
newNode->next = head; struct node *temp = head; while (temp->next != head) { temp =
temp->next;
}
temp->next = newNode; head = newNode;
}}
// Function to insert a node at the end void insertAtEnd(int data) {
struct node *newNode = createNode(data); if (head == NULL) {
head = newNode;
head->next = head;
} else {
struct node *temp = head;
while (temp->next != head) { temp = temp->next;
}
temp->next = newNode;
newNode->next = head;
}
}
// Function to insert a node at a specified position void insertAtPosition(int data, int position) {
if (position < 1) {
printf("Invalid position\n");
return;
}
if (position == 1) { insertAtBeginning(data); return;

```

```

}
struct node *newNode = createNode(data); struct node *temp = head;
int count = 1;
while (count < position - 1 && temp->next != head) { temp = temp->next;
count++;
}

if (count == position - 1) { newNode->next = temp->next; temp->next = newNode;
} else {
printf("Invalid position\n");
}
}

// Function to delete a node from the beginning void deleteAtBeginning() {
if (head == NULL) {
printf("List is empty\n"); return;
}

if (head->next == head) { free(head);
head = NULL;
} else {
struct node *temp = head; head = head->next;
struct node *last = head;
while (last->next != temp) {
last = last->next;
}
last->next = head;
free(temp);
}
}

// Function to delete a node from the end void deleteAtEnd() {

if (head == NULL) {
printf("List is empty\n");
return;
}
if (head->next == head) {
free(head);
head = NULL;
} else {
struct node *temp = head;
while (temp->next->next != head) {
temp = temp->next;

```

```
}  
free(temp->next);  
temp->next = head;  
}  
}
```

```
// Function to delete a node at a specified position void deleteAtPosition(int position) {  
if (position < 1) {  
printf("Invalid position\n");  
return;  
}  

```

```
if (position == 1) {  
deleteAtBeginning();  
return;  
}  
struct node *temp = head;  
int count = 1;  
while (count < position - 1 && temp->next != head) {  
temp = temp->next;  
count++;  
}  

```

```
if (count == position - 1) {  
struct node *toDelete = temp->next;  
temp->next = toDelete->next;  
free(toDelete);  
}  
else {  
printf("Invalid position\n");  
}  
}
```

```
// Function to display the circular linked list void display() {
```

```
if (head == NULL) {  
printf("List is empty\n"); return;  
}  
struct node *temp = head;  
do {  
printf("%d ", temp->data); temp = temp->next;  
} while (temp != head);  
printf("\n");
```

```

}
int main() {
int choice, data, position;
while (1) {
printf("\nCircular Linked List Operations:\n"); printf("1. Create List\n");
printf("2. Insert at Beginning\n");
printf("3. Insert at End\n");
printf("4. Insert at Position\n");
printf("5. Delete at Beginning\n");
printf("6. Delete at End\n");
printf("7. Delete at Position\n");
printf("8. Display\n");
printf("9. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice) {
case 1:
printf("Enter data for the first node: "); scanf("%d", &data);
createList(data);
break;
case 2:
printf("Enter data to insert at beginning: "); scanf("%d", &data);
insertAtBeginning(data);
break;
case 3:
printf("Enter data to insert at end: "); scanf("%d", &data); insertAtEnd(data);
break;
case 4:
printf("Enter data to insert: ");
scanf("%d", &data);
printf("Enter position: ");
scanf("%d", &position);

insertAtPosition(data, position);
break;
case 5: deleteAtBeginning();
break;
case 6: deleteAtEnd();
break;
case 7:
printf("Enter position to delete: ");
scanf("%d", &position); deleteAtPosition(position);
break;
case 8:

```

```
display();  
break;  
case 9:  
exit(0);  
default:  
printf("Invalid choice\n");  
}  
}  
return 0;  
}
```